

GROUP 28

Laksh Jain – 23110185

Tanish Yelgoe – 23110328

Github: [Jain-Laksh/CS203-Lab-06](#)

SECTION-1 CODE

1] Import the iris dataset

```
# loading the dataset
iris_dataset = load_iris()
X = iris_dataset.data
y = iris_dataset.target
print(X.shape, y.shape)

... (150, 4) (150,)

# print the feature and target names
print(iris_dataset.feature_names)
print(iris_dataset.target_names)

... ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
['setosa' 'versicolor' 'virginica']
```

2] One-hot encoding the labels and normalizing the feature values

```
# One hot encoding of the target
y = OneHotEncoder().fit_transform(y.reshape(-1, 1)).toarray()
print(y.shape)

... (150, 3)
```

```
# Normalize feature values to [0,1] using standard scaling
X = MinMaxScaler().fit_transform(X)
print(X)

[[0.22222222 0.625      0.06779661 0.04166667]
 [0.16666667 0.41666667 0.06779661 0.04166667]
 [0.11111111 0.5       0.05084746 0.04166667]
 [0.08333333 0.45833333 0.08474576 0.04166667]
 [0.19444444 0.66666667 0.06779661 0.04166667]
 [0.30555556 0.79166667 0.11864407 0.125     ]]
```

3] Splitting the dataset

```
# Split the dataset into training, validation and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=5, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.125, random_state=5)
```

4] Initializing wandb

```
import wandb

wandb.init(project="iris-classification", name="epoch=50")
```

5] Initializing the model

```

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(4, 16)
        self.fc2 = nn.Linear(16, 3)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        logits = self.fc2(x)
        x = torch.softmax(logits, dim=1)
        return logits, x

mlp = MLP()

```

6] Defining the training loop

```

# Training loop
loss_func = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(mlp.parameters(), lr=0.001)
n_epochs = 50
for epoch in range(n_epochs):
    mlp.train()
    total_loss = 0
    for i, batch in enumerate(trainloader):
        optimizer.zero_grad()
        logits, y_pred = mlp(torch.tensor(batch[0], dtype=torch.float32))
        loss = loss_func(logits, torch.argmax(torch.tensor(batch[1], dtype=torch.float32), dim=1))
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    # Tracking training loss
    train_loss.append(total_loss)
    print(f'Epoch {epoch+1}/{n_epochs} Train Loss: {total_loss}')

    # Tracking validation loss
    logits, y_pred_val = mlp(torch.tensor(X_val, dtype=torch.float32))
    loss_val = loss_func(logits, torch.argmax(torch.tensor(y_val, dtype=torch.float32), dim=1))
    val_loss.append(loss_val.item())
    print(f'Epoch {epoch+1}/{n_epochs} Validation Loss: {loss_val.item()}')

    # Log training and validation loss
    wandb.log({"Epoch": epoch + 1, "Train Loss": total_loss, "Val Loss": loss_val})

```

7] Adding the hyperparameters and model architecture to wandb

```

wandb.config.update({
    "model_architecture": {
        "num_layers": 3,
        "neurons_per_layer": [4, 16, 3],
        "activation_functions": ["ReLU", "SoftMax"]
    },
    "hyperparameters": {
        "learning_rate": 0.001,
        "batch_size": 32,
        "num_epochs": 50
    }
})

```

8] Calculating the metrics and adding them to wandb

```
ACCURACY
Accuracy: 0.7666666666666667
```

```
PRECISION
Precision for setosa: 1.0000
Precision for versicolor: 1.0000
Precision for virginica: 0.5882
Average Precision: 0.8627
```

```
RECALL
Recall for setosa: 1.0000
Recall for versicolor: 0.3000
Recall for virginica: 1.0000
Average Recall: 0.7667
```

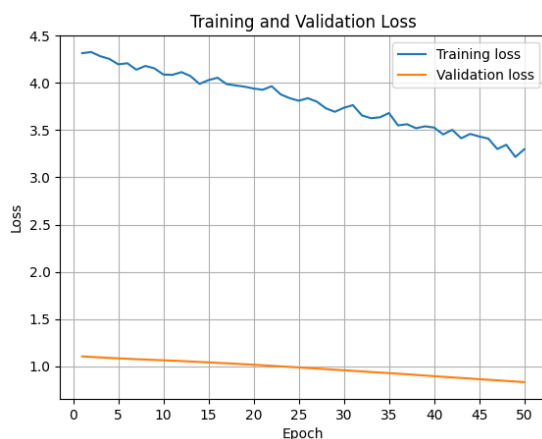
```
F1 SCORE
F1 Score for setosa: 1.0000
F1 Score for versicolor: 0.4615
F1 Score for virginica: 0.7407
Average F1 Score: 0.7341
```

```
CONFUSION MATRIX
Confusion Matrix:
[[10  0  0]
 [ 0  3  7]
 [ 0  0 10]]
```

```
wandb.log({
    "Accuracy": accuracy,
    "Precision": precision.mean(),
    "Recall": recall.mean(),
    "F1 score": f1.mean()
})
```

9] Plot the training and validation losses

```
# Plot training and validation loss
plt.plot(range(1,n_epochs+1),train_loss, label='Training loss')
plt.plot(range(1,n_epochs+1),val_loss, label='Validation loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.xticks(range(0,n_epochs+1,5))
plt.grid()
plt.legend()
plt.show()
```

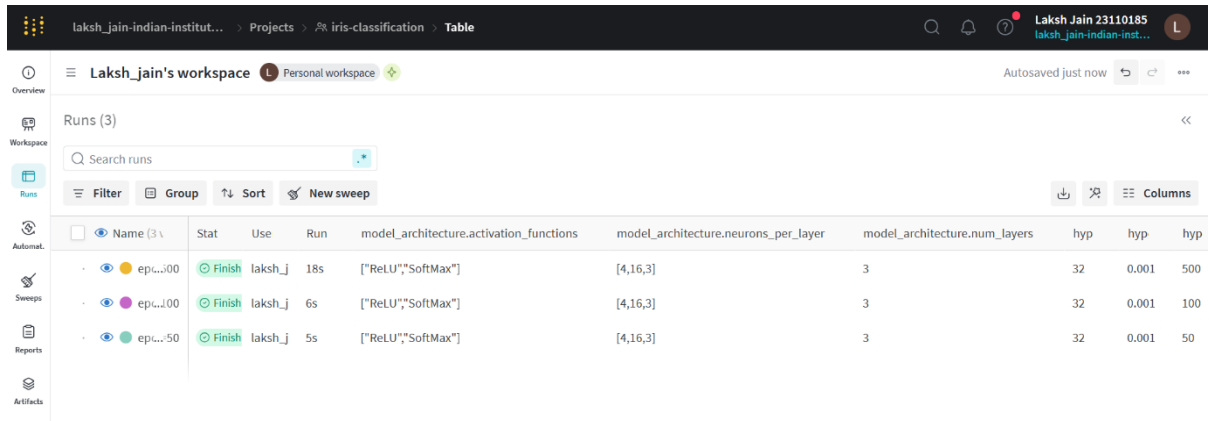


10] Ending wand

```
wandb.finish()
```

SECTION-1 SCREENSHOTS

1] Model Architecture



laksh_jain-indian-institut... > Projects > iris-classification > Table

Laksh Jain 23110185
laksh_jain-indian-inst...

Autosaved just now

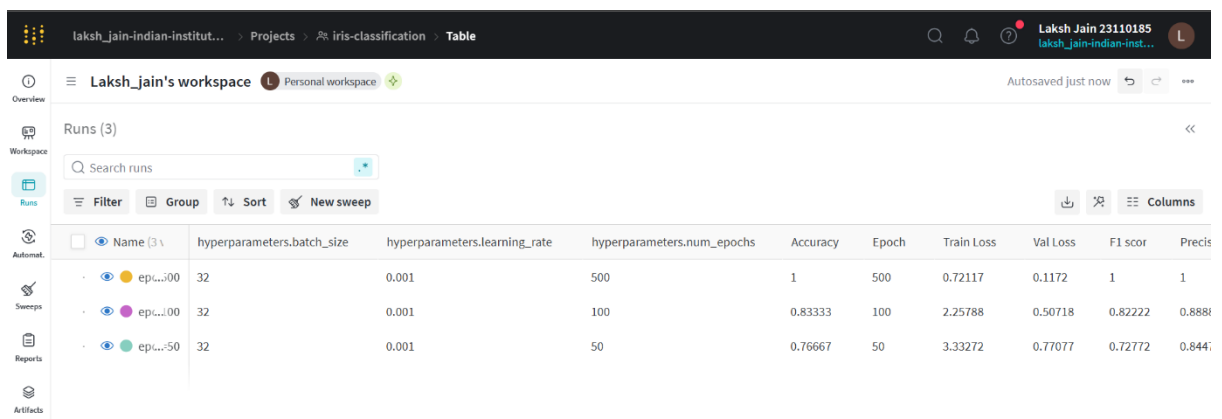
Runs (3)

Search runs

Filter Group Sort New sweep

Name (3)	Stat	Use	Run	model_architecture.activation_functions	model_architecture.neurons_per_layer	model_architecture.num_layers	hyp	hyp	hyp
epi...00	Finish	laksh_j	18s	["ReLU","SoftMax"]	[4,16,3]	3	32	0.001	500
epi...00	Finish	laksh_j	6s	["ReLU","SoftMax"]	[4,16,3]	3	32	0.001	100
epi...50	Finish	laksh_j	5s	["ReLU","SoftMax"]	[4,16,3]	3	32	0.001	50

2] Hyperparameters



laksh_jain-indian-institut... > Projects > iris-classification > Table

Laksh Jain 23110185
laksh_jain-indian-inst...

Autosaved just now

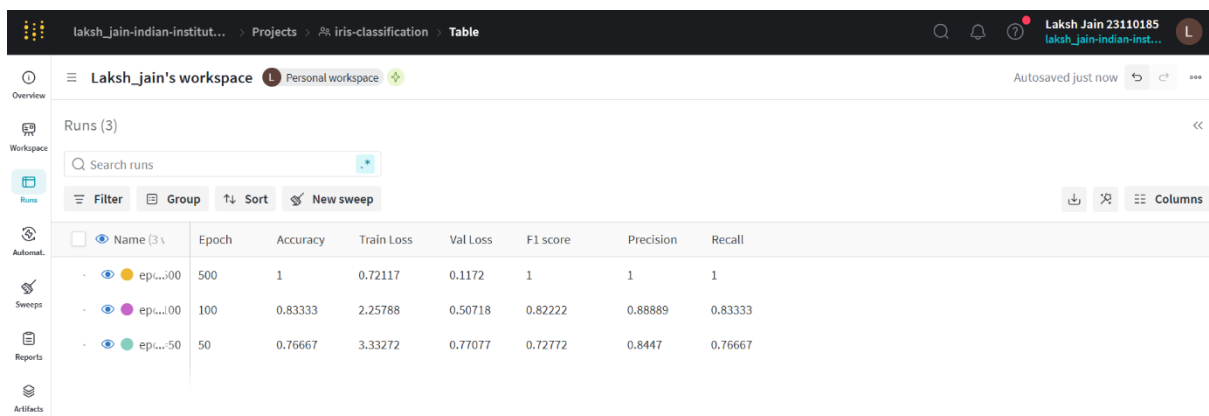
Runs (3)

Search runs

Filter Group Sort New sweep

Name (3)	hyperparameters.batch_size	hyperparameters.learning_rate	hyperparameters.num_epochs	Accuracy	Epoch	Train Loss	Val Loss	F1 scor	Precis
epi...00	32	0.001	500	1	500	0.72117	0.1172	1	1
epi...00	32	0.001	100	0.83333	100	2.25788	0.50718	0.82222	0.888
epi...50	32	0.001	50	0.76667	50	3.33272	0.77077	0.72772	0.844

3] Final evaluation metrics



laksh_jain-indian-institut... > Projects > iris-classification > Table

Laksh Jain 23110185
laksh_jain-indian-inst...

Autosaved just now

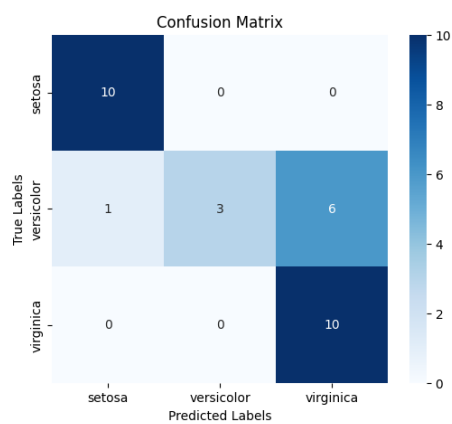
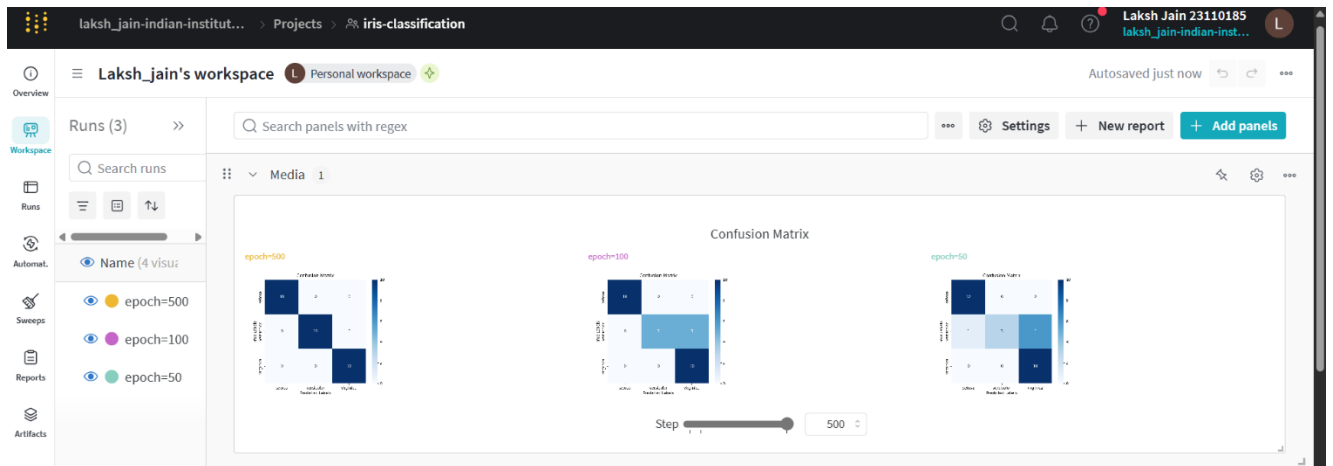
Runs (3)

Search runs

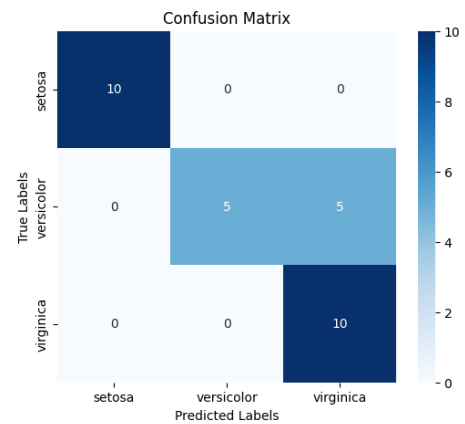
Filter Group Sort New sweep

Name (3)	Epoch	Accuracy	Train Loss	Val Loss	F1 score	Precision	Recall
epi...00	500	1	0.72117	0.1172	1	1	1
epi...00	100	0.83333	2.25788	0.50718	0.82222	0.88889	0.83333
epi...50	50	0.76667	3.33272	0.77077	0.72772	0.8447	0.76667

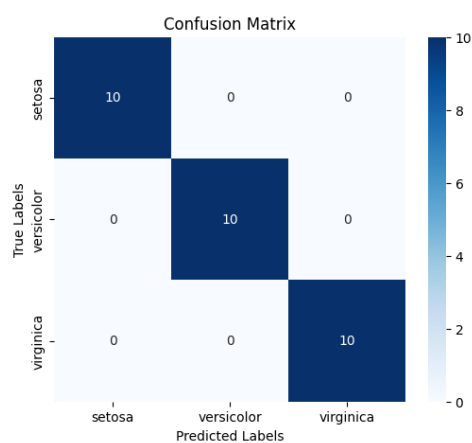
4] Confusion Matrix



For epoch = 50

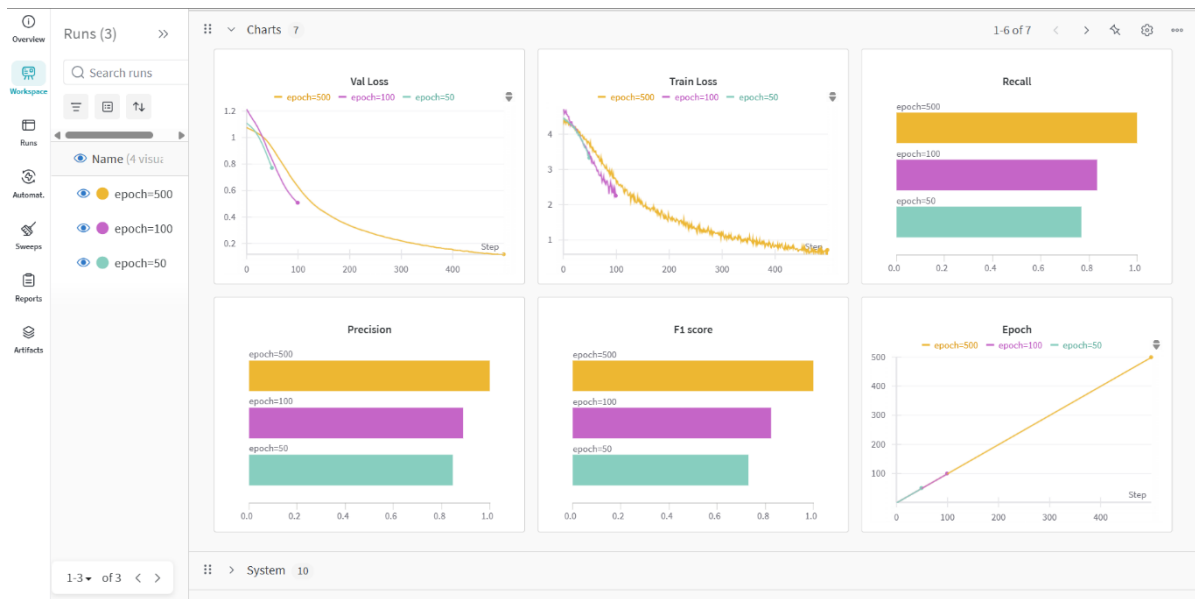


For epoch = 100

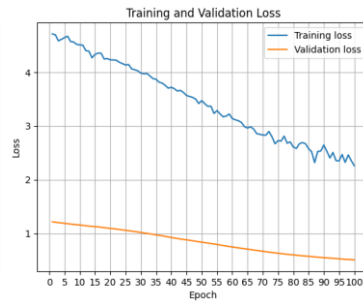


For epoch = 500

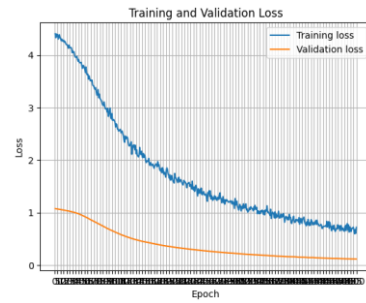
5] Logged Metrics and Loss Curves



For epoch = 50



For epoch = 100



For epoch = 500

SECTION-2

TASK1

Dividing the dataset into batches of size 2 and 4

```
# Divide the trainset into batches
trainset = torch.utils.data.TensorDataset(X_train, y_train)
trainloader_2 = torch.utils.data.DataLoader(dataset=trainset, batch_size=2, shuffle=True)
```

✓ 0.0s

```
# Divide the trainset into batches
trainloader_4 = torch.utils.data.DataLoader(dataset=trainset, batch_size=4, shuffle=True)
```

✓ 0.0s

Initializing the model with same architecture as used in section 1

```
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(4, 16)
        self.fc2 = nn.Linear(16, 3)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        logits = self.fc2(x)
        x = torch.softmax(logits, dim=1)
        return logits, x
```

✓ 0.0s

```
learning_rates = [0.001, 0.00001]
epochs = [1,3,5]
```

✓ 0.0s

For Batch Size 2:

```
# Training loop
loss_func = nn.CrossEntropyLoss()

for lr in learning_rates:
    for ep_num in epochs:
        train_loss = []
        val_loss = []
        mlp = MLP()
        optimizer = torch.optim.Adam(mlp.parameters(), lr=lr)
        n_epochs = ep_num
        for epoch in range(n_epochs):
```

```

        mlp.train()
        total_loss = 0
        for i, batch in enumerate(trainloader_2):
            optimizer.zero_grad()
            logits, y_pred = mlp(torch.tensor(batch[0],
dtype=torch.float32))
            loss = loss_func(logits, torch.argmax(torch.tensor(batch[1],
dtype=torch.float32), dim=1))
            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        # Tracking training loss
        train_loss.append(total_loss)
        print(f'Epoch {epoch+1}/{n_epochs} Train Loss: {total_loss}')

        # Tracking validation loss
        logits, y_pred_val = mlp(torch.tensor(X_val, dtype=torch.float32))
        loss_val = loss_func(logits, torch.argmax(torch.tensor(y_val,
dtype=torch.float32), dim=1))
        val_loss.append(loss_val.item())
        print(f'Epoch {epoch+1}/{n_epochs} Validation Loss:
{loss_val.item()}')

        # Test the model on the test set
        mlp.eval()
        logits_test, y_pred_test = mlp(torch.tensor(X_test,
dtype=torch.float32))
        y_pred_test = torch.argmax(y_pred_test, dim=1).detach().numpy()
        y_test_i = torch.argmax(torch.tensor(y_test, dtype=torch.float32),
dim=1).detach().numpy()

        # compute accuracy
        print("ACCURACY")
        accuracy = accuracy_score(y_test_i, y_pred_test)
        print(f'Accuracy for batch size 2, learning rate {lr} and num epoch
{ep_num}: {accuracy}')
        # compute f1 score
        print(f"\nF1 SCORE FOR BATCH SIZE 2, LEARNING RATE {lr} AND {ep_num}
EPOCHS")
        f1 = f1_score(y_test_i, y_pred_test, average=None)
        print(f'F1 Score for {iris_dataset.target_names[0]}: {f1[0]:.4f}')
        print(f'F1 Score for {iris_dataset.target_names[1]}: {f1[1]:.4f}')
        print(f'F1 Score for {iris_dataset.target_names[2]}: {f1[2]:.4f}')
        print(f'Average F1 Score: {f1.mean():.4f}')

        # compute confusion matrix

```



```

        print(f"\nCONFUSION MATRIX FOR BATCH SIZE 2, LEARNING RATE {lr} AND
{ep_num} EPOCHS")
        confusion_mat = confusion_matrix(y_test_i, y_pred_test)
        print(f'Confusion Matrix: \n{confusion_mat}')

        # visualize confusion matrix using matplotlib
        class_labels = ['setosa' , 'versicolor' , 'virginica']
        plt.figure(figsize=(6, 5))
        sns.heatmap(confusion_mat, annot=True, fmt='d', cmap='Blues',
                    xticklabels=class_labels, yticklabels=class_labels)

        plt.xlabel('Predicted Labels')
        plt.ylabel('True Labels')
        plt.title('Confusion Matrix')
        plt.show()

        # testing the model on 5 values from test dataset
        print(f"\n TESTING THE MODEL ON 5 VALUES FROM TEST DATASET")
        print(f"Model parameters: \nBatch Size = 2 \nLearning Rate = {lr}
\nEpochs = {ep_num}")
        for i in range(5):
            print(f"Example {i}: {X_test[i]}")
            print(f'Actual Label: {iris_dataset.target_names[y_test_i[i]]}')
            print(f'Predicted Label:
{iris_dataset.target_names[y_pred_test[i]]}')

```

A) For learning rate = 0.001 and 1 epoch

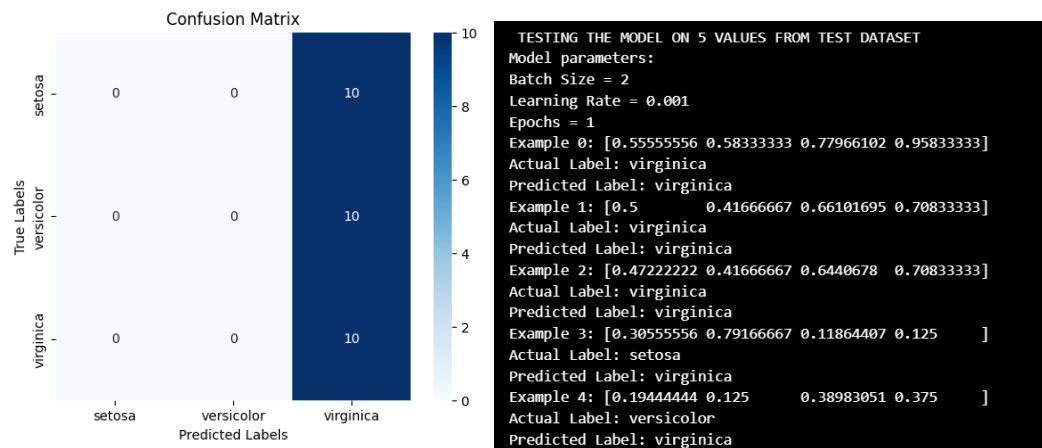
```

Epoch 1/1 Train Loss: 59.262316942214966
Epoch 1/1 Validation Loss: 1.119890809059143
ACCURACY
Accuracy for batch size 2, learning rate 0.001 and num epoch 1: 0.3333333333333333

F1 SCORE FOR BATCH SIZE 2, LEARNING RATE 0.001 AND 1 EPOCHS
F1 Score for setosa: 0.0000
F1 Score for versicolor: 0.0000
F1 Score for virginica: 0.5000
Average F1 Score: 0.1667

CONFUSION MATRIX FOR BATCH SIZE 2, LEARNING RATE 0.001 AND 1 EPOCHS
Confusion Matrix:
[[ 0  0 10]
 [ 0  0 10]
 [ 0  0 10]]

```

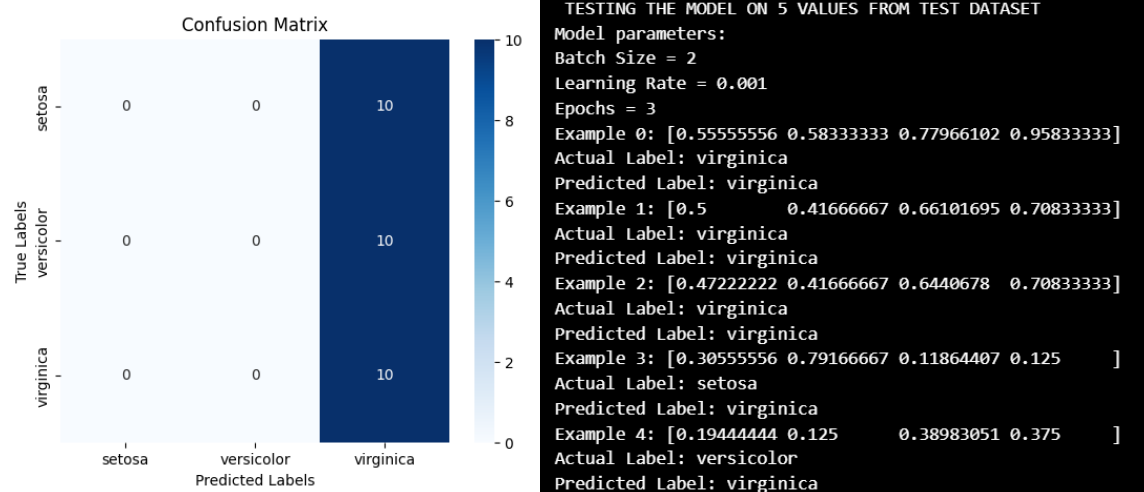


B) For learning rate = 0.001 and 3 epochs

```
Epoch 1/3 Train Loss: 59.10639727115631
Epoch 1/3 Validation Loss: 1.129027009010315
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\3896208356.py:16: UserWarning: To
  logits, y_pred = mlp(torch.tensor(batch[0], dtype=torch.float32))
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\3896208356.py:17: UserWarning: To
  loss = loss_func(logits, torch.argmax(torch.tensor(batch[1], dtype=torch.float32),
Epoch 2/3 Train Loss: 57.318559885025024
Epoch 2/3 Validation Loss: 1.1057288646697998
Epoch 3/3 Train Loss: 55.62063443660736
Epoch 3/3 Validation Loss: 1.0795611143112183
ACCURACY
Accuracy for batch size 2, learning rate 0.001 and num epoch 3: 0.3333333333333333

F1 SCORE FOR BATCH SIZE 2, LEARNING RATE 0.001 AND 3 EPOCHS
F1 Score for setosa: 0.0000
F1 Score for versicolor: 0.0000
F1 Score for virginica: 0.5000
Average F1 Score: 0.1667

CONFUSION MATRIX FOR BATCH SIZE 2, LEARNING RATE 0.001 AND 3 EPOCHS
Confusion Matrix:
[[ 0  0 10]
 [ 0  0 10]
 [ 0  0 10]]
```



C] For learning rate = 0.001 and 5 epochs

```
Epoch 1/5 Train Loss: 56.57544416189194
Epoch 1/5 Validation Loss: 1.10542631149292
Epoch 2/5 Train Loss: 54.47329157590866
Epoch 2/5 Validation Loss: 1.0522749423980713
Epoch 3/5 Train Loss: 52.61577343940735
Epoch 3/5 Validation Loss: 1.0112923383712769
Epoch 4/5 Train Loss: 50.586233377456665
Epoch 4/5 Validation Loss: 0.9593207240104675
Epoch 5/5 Train Loss: 48.162006080150604
Epoch 5/5 Validation Loss: 0.8971816897392273
ACCURACY
Accuracy for batch size 2, learning rate 0.001 and num epoch 5: 0.8666666666666667
```

F1 SCORE FOR BATCH SIZE 2, LEARNING RATE 0.001 AND 5 EPOCHS

F1 Score for setosa: 0.9474

F1 Score for versicolor: 0.7778

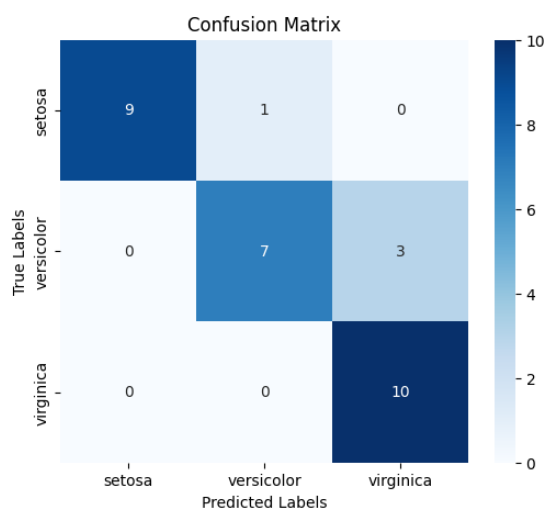
F1 Score for virginica: 0.8696

Average F1 Score: 0.8649

CONFUSION MATRIX FOR BATCH SIZE 2, LEARNING RATE 0.001 AND 5 EPOCHS

Confusion Matrix:

```
[[ 9  1  0]
 [ 0  7  3]
 [ 0  0 10]]
```



TESTING THE MODEL ON 5 VALUES FROM TEST DATASET

Model parameters:
Batch Size = 2
Learning Rate = 0.001
Epochs = 5

Example 0: [0.55555556 0.58333333 0.77966102 0.95833333]
Actual Label: virginica
Predicted Label: virginica

Example 1: [0.5 0.41666667 0.66101695 0.70833333]
Actual Label: virginica
Predicted Label: virginica

Example 2: [0.47222222 0.41666667 0.6440678 0.70833333]
Actual Label: virginica
Predicted Label: virginica

Example 3: [0.30555556 0.79166667 0.11864407 0.125]
Actual Label: setosa
Predicted Label: setosa

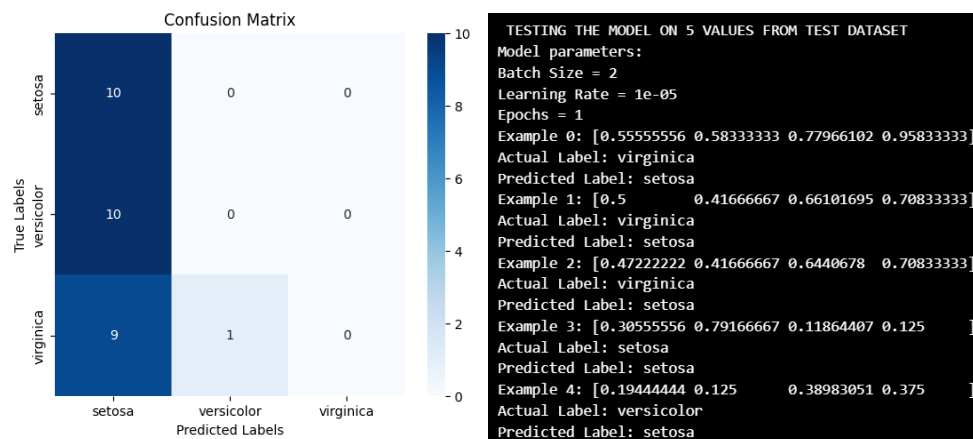
Example 4: [0.19444444 0.125 0.38983051 0.375]
Actual Label: versicolor
Predicted Label: versicolor

D] For learning rate = 0.00001 and 1 epochs

```
Epoch 1/1 Train Loss: 58.23640513420105
Epoch 1/1 Validation Loss: 1.0809093713760376
ACCURACY
Accuracy for batch size 2, learning rate 1e-05 and num epoch 1: 0.3333333333333333

F1 SCORE FOR BATCH SIZE 2, LEARNING RATE 1e-05 AND 1 EPOCHS
F1 Score for setosa: 0.5128
F1 Score for versicolor: 0.0000
F1 Score for virginica: 0.0000
Average F1 Score: 0.1709

CONFUSION MATRIX FOR BATCH SIZE 2, LEARNING RATE 1e-05 AND 1 EPOCHS
Confusion Matrix:
[[10  0  0]
 [10  0  0]
 [ 9  1  0]]
```

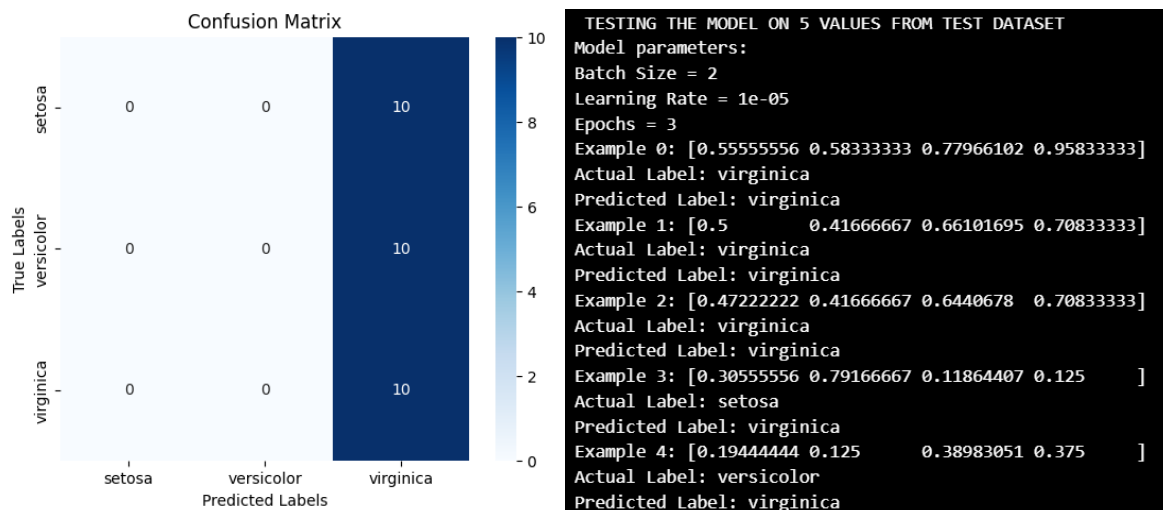


E] For learning rate = 0.00001 and 3 epochs

```
Epoch 1/3 Train Loss: 61.361304581165314
Epoch 1/3 Validation Loss: 1.2021234035491943
C:\Users\1511\AppData\Local\Temp\ipykernel_31296\3896208356.py:16: UserWarning: To
logits, y_pred = mlp(torch.tensor(batch[0], dtype=torch.float32))
C:\Users\1511\AppData\Local\Temp\ipykernel_31296\3896208356.py:17: UserWarning: To
loss = loss_func(logits, torch.argmax(torch.tensor(batch[1], dtype=torch.float32))
Epoch 2/3 Train Loss: 61.16252517700195
Epoch 2/3 Validation Loss: 1.201646089553833
Epoch 3/3 Train Loss: 61.206918478012085
Epoch 3/3 Validation Loss: 1.201203465461731
ACCURACY
Accuracy for batch size 2, learning rate 1e-05 and num epoch 3: 0.3333333333333333

F1 SCORE FOR BATCH SIZE 2, LEARNING RATE 1e-05 AND 3 EPOCHS
F1 Score for setosa: 0.0000
F1 Score for versicolor: 0.0000
F1 Score for virginica: 0.5000
Average F1 Score: 0.1667

CONFUSION MATRIX FOR BATCH SIZE 2, LEARNING RATE 1e-05 AND 3 EPOCHS
Confusion Matrix:
[[ 0  0 10]
 [ 0  0 10]
 [ 0  0 10]]
```

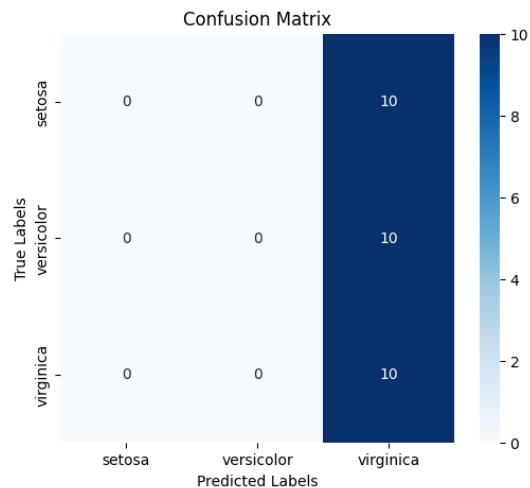


F] For learning rate = 0.00001 and 5 epochs

```
Epoch 1/5 Train Loss: 57.76787328720093
Epoch 1/5 Validation Loss: 1.1455093622207642
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\3896208356.py:16: UserWarning: To
logits, y_pred = mlp(torch.tensor(batch[0], dtype=torch.float32))
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\3896208356.py:17: UserWarning: To
loss = loss_func(logits, torch.argmax(torch.tensor(batch[1], dtype=torch.float32),
Epoch 2/5 Train Loss: 57.835738599300385
Epoch 2/5 Validation Loss: 1.1452580690383911
Epoch 3/5 Train Loss: 57.74388438463211
Epoch 3/5 Validation Loss: 1.144958734512329
Epoch 4/5 Train Loss: 57.55912774801254
Epoch 4/5 Validation Loss: 1.1446610689163208
Epoch 5/5 Train Loss: 57.71918797492981
Epoch 5/5 Validation Loss: 1.1443449258804321
ACCURACY
Accuracy for batch size 2, learning rate 1e-05 and num epoch 5: 0.3333333333333333

F1 SCORE FOR BATCH SIZE 2, LEARNING RATE 1e-05 AND 5 EPOCHS
F1 Score for setosa: 0.0000
F1 Score for versicolor: 0.0000
F1 Score for virginica: 0.5000
Average F1 Score: 0.1667

CONFUSION MATRIX FOR BATCH SIZE 2, LEARNING RATE 1e-05 AND 5 EPOCHS
Confusion Matrix:
[[ 0  0 10]
 [ 0  0 10]
 [ 0  0 10]]
```



TESTING THE MODEL ON 5 VALUES FROM TEST DATASET

Model parameters:
 Batch Size = 2
 Learning Rate = 1e-05
 Epochs = 5

Example 0: [0.55555556 0.58333333 0.77966102 0.95833333]
 Actual Label: virginica
 Predicted Label: virginica

Example 1: [0.5 0.41666667 0.66101695 0.70833333]
 Actual Label: virginica
 Predicted Label: virginica

Example 2: [0.47222222 0.41666667 0.6440678 0.70833333]
 Actual Label: virginica
 Predicted Label: virginica

Example 3: [0.30555556 0.79166667 0.11864407 0.125]
 Actual Label: setosa
 Predicted Label: virginica

Example 4: [0.19444444 0.125 0.38983051 0.375]
 Actual Label: versicolor
 Predicted Label: virginica

For Batch Size 4:

```
# Lists for storing training and validation loss
train_loss = []
val_loss = []

# Training loop
loss_func = nn.CrossEntropyLoss()

for lr in learning_rates:
    for ep_num in epochs:
        mlp = MLP()
        optimizer = torch.optim.Adam(mlp.parameters(), lr=lr)
        n_epochs = ep_num
        for epoch in range(n_epochs):
            mlp.train()
            total_loss = 0
            for i, batch in enumerate(trainloader_4):
                optimizer.zero_grad()
                logits, y_pred = mlp(torch.tensor(batch[0],
dtype=torch.float32))
                loss = loss_func(logits, torch.argmax(torch.tensor(batch[1],
dtype=torch.float32), dim=1))
                loss.backward()
                optimizer.step()
                total_loss += loss.item()
```

```

        # Tracking training loss
        train_loss.append(total_loss)
        print(f'Epoch {epoch+1}/{n_epochs} Train Loss: {total_loss}')

        # Tracking validation loss
        logits, y_pred_val = mlp(torch.tensor(X_val, dtype=torch.float32))
        loss_val = loss_func(logits, torch.argmax(torch.tensor(y_val,
dtype=torch.float32), dim=1))
        val_loss.append(loss_val.item())
        print(f'Epoch {epoch+1}/{n_epochs} Validation Loss:
{loss_val.item()}')

    # Test the model on the test set
    mlp.eval()
    logits_test, y_pred_test = mlp(torch.tensor(X_test,
dtype=torch.float32))
    y_pred_test = torch.argmax(y_pred_test, dim=1).detach().numpy()
    y_test_i = torch.argmax(torch.tensor(y_test, dtype=torch.float32),
dim=1).detach().numpy()

    # compute accuracy
    print("ACCURACY")
    accuracy = accuracy_score(y_test_i, y_pred_test)
    print(f'Accuracy for batch size 4, learning rate {lr} and num epoch
{ep_num}: {accuracy}')
    # compute f1 score
    print(f"\nF1 SCORE FOR BATCH SIZE 4, LEARNING RATE {lr} AND {ep_num}
EPOCHS")
    f1 = f1_score(y_test_i, y_pred_test, average=None)
    print(f'F1 Score for {iris_dataset.target_names[0]}: {f1[0]:.4f}')
    print(f'F1 Score for {iris_dataset.target_names[1]}: {f1[1]:.4f}')
    print(f'F1 Score for {iris_dataset.target_names[2]}: {f1[2]:.4f}')
    print(f'Average F1 Score: {f1.mean():.4f}')

    # compute confusion matrix
    print(f"\nCONFUSION MATRIX FOR BATCH SIZE 4, LEARNING RATE {lr} AND
{ep_num} EPOCHS")
    confusion_mat = confusion_matrix(y_test_i, y_pred_test)
    print(f'Confusion Matrix: \n{confusion_mat}')

    # visualize confusion matrix using matplotlib
    class_labels = ['setosa', 'versicolor', 'virginica']
    plt.figure(figsize=(6, 5))
    sns.heatmap(confusion_mat, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_labels, yticklabels=class_labels)

    plt.xlabel('Predicted Labels')
    plt.ylabel('True Labels')

```

```

plt.title('Confusion Matrix')
plt.show()

# testing the model on 5 values from test dataset
print(f"\n TESTING THE MODEL ON 5 VALUES FROM TEST DATASET")
print(f"Model parameters: \nBatch Size = 4 \nLearning Rate = {lr} \nEpochs = {ep_num}")
for i in range(5):
    print(f"Example {i}: {X_test[i]}")
    print(f'Actual Label: {iris_dataset.target_names[y_test_i[i]]}')
    print(f'Predicted Label: {iris_dataset.target_names[y_pred_test[i]]}')

```

A) For learning rate = 0.001 and 1 epoch

```

Epoch 1/1 Train Loss: 29.922856152057648
Epoch 1/1 Validation Loss: 1.1012022495269775
ACCURACY
Accuracy for batch size 4, learning rate 0.001 and num epoch 1: 0.3333333333333333

```

F1 SCORE FOR BATCH SIZE 4, LEARNING RATE 0.001 AND 1 EPOCHS

```

F1 Score for setosa: 0.0000
F1 Score for versicolor: 0.5000
F1 Score for virginica: 0.0000
Average F1 Score: 0.1667

```

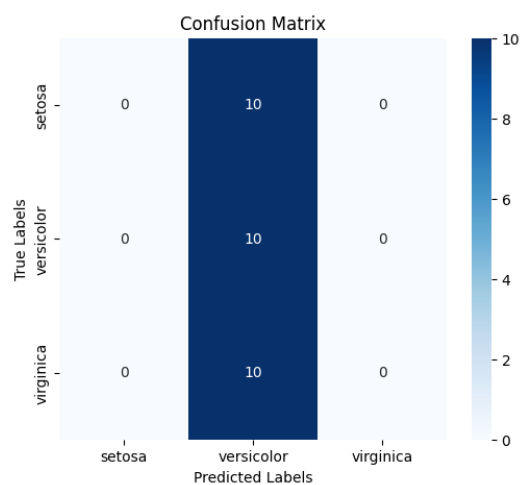
CONFUSION MATRIX FOR BATCH SIZE 4, LEARNING RATE 0.001 AND 1 EPOCHS

Confusion Matrix:

```

[[ 0 10  0]
 [ 0 10  0]
 [ 0 10  0]]

```



```

TESTING THE MODEL ON 5 VALUES FROM TEST DATASET
Model parameters:
Batch Size = 4
Learning Rate = 0.001
Epochs = 1
Example 0: [0.55555556 0.58333333 0.77966102 0.95833333]
Actual Label: virginica
Predicted Label: versicolor
Example 1: [0.5          0.41666667 0.66101695 0.70833333]
Actual Label: virginica
Predicted Label: versicolor
Example 2: [0.47222222 0.41666667 0.6440678  0.70833333]
Actual Label: virginica
Predicted Label: versicolor
Example 3: [0.30555556 0.79166667 0.11864407 0.125      ]
Actual Label: setosa
Predicted Label: versicolor
Example 4: [0.19444444 0.125          0.38983051 0.375      ]
Actual Label: versicolor
Predicted Label: versicolor

```


B] For learning rate = 0.001 and 3 epochs

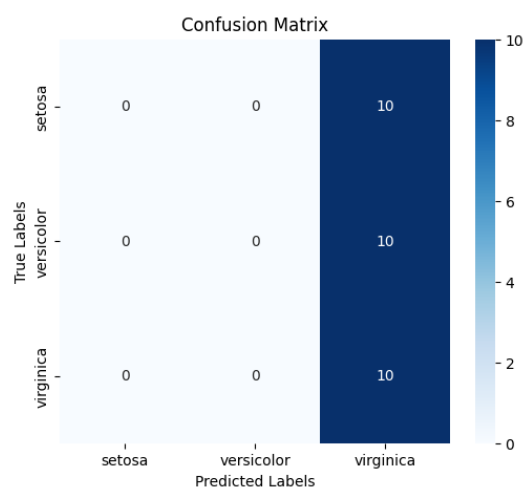
```
Epoch 1/3 Train Loss: 30.019225358963013
Epoch 1/3 Validation Loss: 1.1269892454147339
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\2099292546.py:18: UserWarning: T
  logits, y_pred = mlp(torch.tensor(batch[0], dtype=torch.float32))
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\2099292546.py:19: UserWarning: T
  loss = loss_func(logits, torch.argmax(torch.tensor(batch[1], dtype=torch.float32
Epoch 2/3 Train Loss: 29.599234461784363
Epoch 2/3 Validation Loss: 1.113968014717102
Epoch 3/3 Train Loss: 28.877332627773285
Epoch 3/3 Validation Loss: 1.0985313653945923
ACCURACY
Accuracy for batch size 4, learning rate 0.001 and num epoch 3: 0.3333333333333333

F1 SCORE FOR BATCH SIZE 4, LEARNING RATE 0.001 AND 3 EPOCHS
F1 Score for setosa: 0.0000
F1 Score for versicolor: 0.0000
F1 Score for virginica: 0.5000
Average F1 Score: 0.1667
```

CONFUSION MATRIX FOR BATCH SIZE 4, LEARNING RATE 0.001 AND 3 EPOCHS

Confusion Matrix:

```
[[ 0  0 10]
 [ 0  0 10]
 [ 0  0 10]]
```



TESTING THE MODEL ON 5 VALUES FROM TEST DATASET

Model parameters:
Batch Size = 4
Learning Rate = 0.001
Epochs = 3

Example 0: [0.55555556 0.58333333 0.77966102 0.95833333]
Actual Label: virginica
Predicted Label: virginica

Example 1: [0.5 0.41666667 0.66101695 0.70833333]
Actual Label: virginica
Predicted Label: virginica

Example 2: [0.47222222 0.41666667 0.6440678 0.70833333]
Actual Label: virginica
Predicted Label: virginica

Example 3: [0.30555556 0.79166667 0.11864407 0.125]
Actual Label: setosa
Predicted Label: virginica

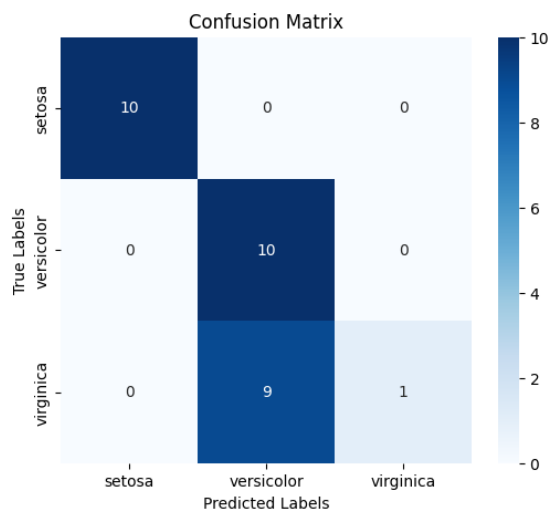
Example 4: [0.19444444 0.125 0.38983051 0.375]
Actual Label: versicolor
Predicted Label: virginica

C] For learning rate = 0.001 and 5 epochs

```
Epoch 1/5 Train Loss: 32.33870869874954
Epoch 1/5 Validation Loss: 1.0824406147003174
Epoch 2/5 Train Loss: 30.549874365329742
Epoch 2/5 Validation Loss: 1.0515210628509521
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\2099292546.py:18: U
logits, y_pred = mlp(torch.tensor(batch[0], dtype=torch.float32))
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\2099292546.py:19: U
loss = loss_func(logits, torch.argmax(torch.tensor(batch[1], dtype=
Epoch 3/5 Train Loss: 29.60483682155609
Epoch 3/5 Validation Loss: 1.0209978818893433
Epoch 4/5 Train Loss: 28.483417332172394
Epoch 4/5 Validation Loss: 0.9915270209312439
Epoch 5/5 Train Loss: 27.425296008586884
Epoch 5/5 Validation Loss: 0.9467567801475525
ACCURACY
Accuracy for batch size 4, learning rate 0.001 and num epoch 5: 0.7

F1 SCORE FOR BATCH SIZE 4, LEARNING RATE 0.001 AND 5 EPOCHS
F1 Score for setosa: 1.0000
F1 Score for versicolor: 0.6897
F1 Score for virginica: 0.1818
Average F1 Score: 0.6238

CONFUSION MATRIX FOR BATCH SIZE 4, LEARNING RATE 0.001 AND 5 EPOCHS
Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  9  1]]
```



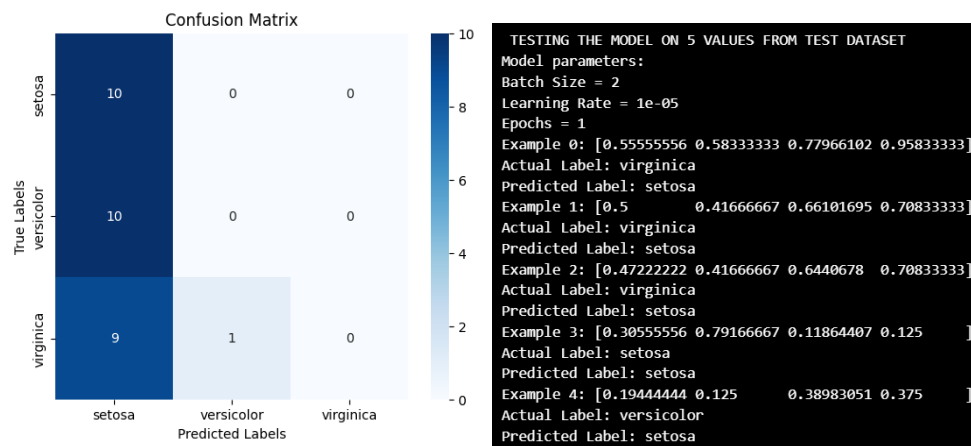
```
TESTING THE MODEL ON 5 VALUES FROM TEST DATASET
Model parameters:
Batch Size = 4
Learning Rate = 0.001
Epochs = 5
Example 0: [0.55555556 0.58333333 0.77966102 0.95833333]
Actual Label: virginica
Predicted Label: versicolor
Example 1: [0.5          0.41666667 0.66101695 0.70833333]
Actual Label: virginica
Predicted Label: versicolor
Example 2: [0.47222222 0.41666667 0.6440678  0.70833333]
Actual Label: virginica
Predicted Label: versicolor
Example 3: [0.30555556 0.79166667 0.11864407 0.125      ]
Actual Label: setosa
Predicted Label: setosa
Example 4: [0.19444444 0.125      0.38983051 0.375      ]
Actual Label: versicolor
Predicted Label: versicolor
```

D] For learning rate = 0.00001 and 1 epochs

```
Epoch 1/1 Train Loss: 58.23640513420105
Epoch 1/1 Validation Loss: 1.0809093713760376
ACCURACY
Accuracy for batch size 2, learning rate 1e-05 and num epoch 1: 0.3333333333333333

F1 SCORE FOR BATCH SIZE 2, LEARNING RATE 1e-05 AND 1 EPOCHS
F1 Score for setosa: 0.5128
F1 Score for versicolor: 0.0000
F1 Score for virginica: 0.0000
Average F1 Score: 0.1709

CONFUSION MATRIX FOR BATCH SIZE 2, LEARNING RATE 1e-05 AND 1 EPOCHS
Confusion Matrix:
[[10  0  0]
 [10  0  0]
 [ 9  1  0]]
```

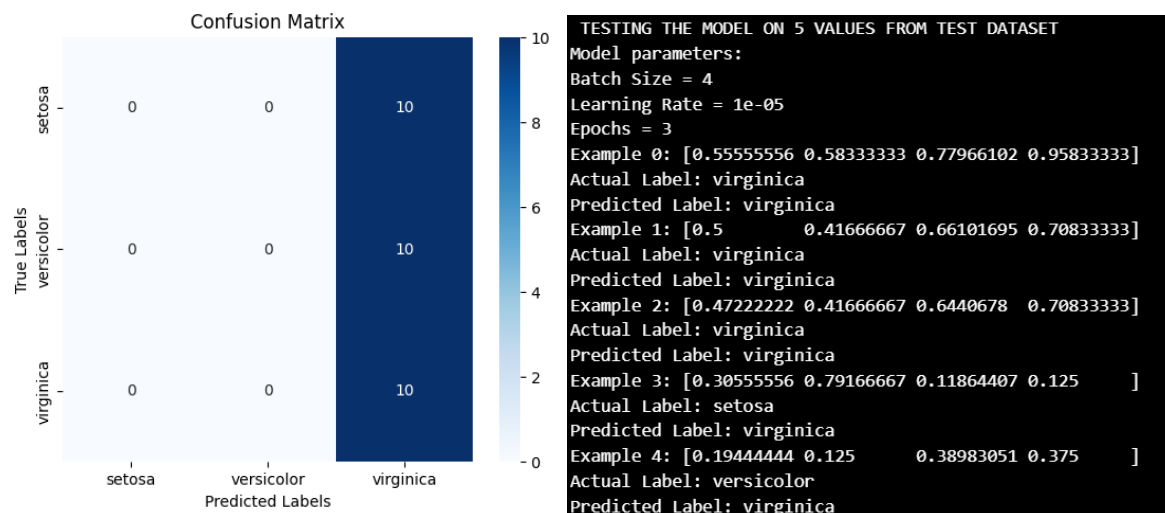


E] For learning rate = 0.00001 and 3 epochs

```
Epoch 1/3 Train Loss: 30.018993973731995
Epoch 1/3 Validation Loss: 1.1786236763000488
Epoch 2/3 Train Loss: 30.21945583820343
Epoch 2/3 Validation Loss: 1.178436517715454
C:\Users\1511\AppData\Local\Temp\ipykernel_31296\2099292546.py:18: UserWarning: To
logits, y_pred = mlp(torch.tensor(batch[0], dtype=torch.float32))
C:\Users\1511\AppData\Local\Temp\ipykernel_31296\2099292546.py:19: UserWarning: To
loss = loss_func(logits, torch.argmax(torch.tensor(batch[1], dtype=torch.float32))
Epoch 3/3 Train Loss: 30.223193407058716
Epoch 3/3 Validation Loss: 1.1782292127609253
ACCURACY
Accuracy for batch size 4, learning rate 1e-05 and num epoch 3: 0.3333333333333333

F1 SCORE FOR BATCH SIZE 4, LEARNING RATE 1e-05 AND 3 EPOCHS
F1 Score for setosa: 0.0000
F1 Score for versicolor: 0.0000
F1 Score for virginica: 0.5000
Average F1 Score: 0.1667

CONFUSION MATRIX FOR BATCH SIZE 4, LEARNING RATE 1e-05 AND 3 EPOCHS
Confusion Matrix:
[[ 0  0 10]
 [ 0  0 10]
 [ 0  0 10]]
```

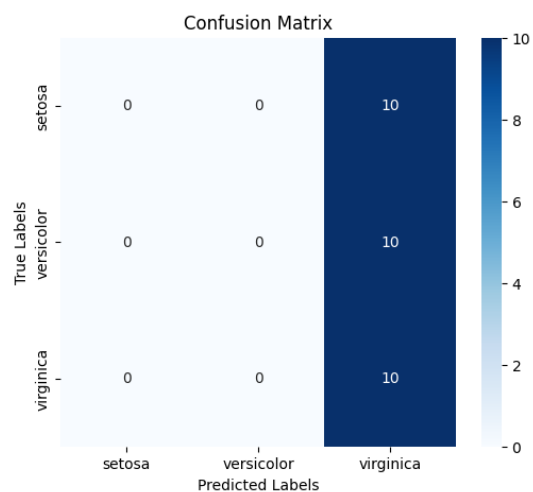


F] For learning rate = 0.00001 and 5 epochs

```
Epoch 1/5 Train Loss: 30.26646637916565
Epoch 1/5 Validation Loss: 1.1437028646469116
Epoch 2/5 Train Loss: 30.26931095123291
Epoch 2/5 Validation Loss: 1.1434669494628906
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\2099292546.py:18: UserWarning: To
logits, y_pred = mlp(torch.tensor(batch[0], dtype=torch.float32))
C:\Users\15111\AppData\Local\Temp\ipykernel_31296\2099292546.py:19: UserWarning: To
loss = loss_func(logits, torch.argmax(torch.tensor(batch[1], dtype=torch.float32),
Epoch 3/5 Train Loss: 29.85843849182129
Epoch 3/5 Validation Loss: 1.1432533264160156
Epoch 4/5 Train Loss: 30.242053985595703
Epoch 4/5 Validation Loss: 1.1430610418319702
Epoch 5/5 Train Loss: 30.17263752222061
Epoch 5/5 Validation Loss: 1.1428176164627075
ACCURACY
Accuracy for batch size 4, learning rate 1e-05 and num epoch 5: 0.3333333333333333

F1 SCORE FOR BATCH SIZE 4, LEARNING RATE 1e-05 AND 5 EPOCHS
F1 Score for setosa: 0.0000
F1 Score for versicolor: 0.0000
F1 Score for virginica: 0.5000
Average F1 Score: 0.1667

CONFUSION MATRIX FOR BATCH SIZE 4, LEARNING RATE 1e-05 AND 5 EPOCHS
Confusion Matrix:
[[ 0  0 10]
 [ 0  0 10]
 [ 0  0 10]]
```



```
TESTING THE MODEL ON 5 VALUES FROM TEST DATASET
Model parameters:
Batch Size = 4
Learning Rate = 1e-05
Epochs = 5
Example 0: [0.55555556 0.58333333 0.77966102 0.95833333]
Actual Label: virginica
Predicted Label: virginica
Example 1: [0.5          0.41666667 0.66101695 0.70833333]
Actual Label: virginica
Predicted Label: virginica
Example 2: [0.47222222 0.41666667 0.6440678  0.70833333]
Actual Label: virginica
Predicted Label: virginica
Example 3: [0.30555556 0.79166667 0.11864407 0.125      ]
Actual Label: setosa
Predicted Label: virginica
Example 4: [0.19444444 0.125      0.38983051 0.375      ]
Actual Label: versicolor
Predicted Label: virginica
```

Task 2_ Part 2

Importing libraries

```
▼ TASK 2

[] import pandas as pd
   from sklearn.model_selection import train_test_split
   from sklearn.metrics import accuracy_score, f1_score
```

Installing autogluon

```
!pip install autogluon

Requirement already satisfied: autogluon in /usr/local/lib/python3.11/dist-packages (1.2)
Requirement already satisfied: autogluon.core==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon.core[all]==1.2->autogluon) (1.2)
Requirement already satisfied: autogluon.features==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon) (1.2)
Requirement already satisfied: autogluon.tabular==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon.tabular[all]==1.2->autogluon) (1.2)
Requirement already satisfied: autogluon.multimodal==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon) (1.2)
Requirement already satisfied: autogluon.timeseries==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon.timeseries[all]==1.2->autogluon) (1.2)
Requirement already satisfied: numpy<2.1.4,>=1.25.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.26.4)
Requirement already satisfied: scipy<1.16,>=1.5.4 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.13.1)
Requirement already satisfied: scikit-learn<1.5.3,>=1.4.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.5.2)
Requirement already satisfied: networkx<4,>=3.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (3.4.2)
Requirement already satisfied: pandas<2.3.0,>=2.0.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (2.2.2)
Requirement already satisfied: tqdm<5,>=4.58 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (4.67.1)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (2.22.3)
Requirement already satisfied: matplotlib<3.11,>=3.7.0 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (3.10.0)
Requirement already satisfied: boto3<2,>=1.18 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.37.1)
Requirement already satisfied: autogluon.common==1.2 in /usr/local/lib/python3.11/dist-packages (from autogluon.core==1.2->autogluon.core[all]==1.2->autogluon) (1.2)
```

Loading the iris dataset and preprocessing

```
import pandas as pd
import autogluon.common as ag
from autogluon.tabular import TabularDataset, TabularPredictor
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

# Load the Iris dataset and construct a DataFrame
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

# Split the data into train, validation, and test subsets
train_val, test_set = train_test_split(df, test_size=0.1, stratify=df['target'], random_state=42)
train_set, val_set = train_test_split(train_val, test_size=0.2 / 0.9, stratify=train_val['target'], random_state=42)

# Convert DataFrames to Autogluon TabularDataset format
train_data = TabularDataset(train_set)
val_data = TabularDataset(val_set)
test_data = TabularDataset(test_set)
true_labels = test_data['target']
```

Creates a predictor function that fits a tabular predictor using hyperparameter tuning based on the search method provided

```
def build_predictor(search_method: str) -> TabularPredictor:
    """
    Constructs and fits a TabularPredictor using hyperparameter tuning.

    Parameters:
        search_method (str): The search strategy for hyperparameter tuning ('random' or 'bayes').

    Returns:
        A fitted TabularPredictor.
    """
    predictor = TabularPredictor(label='target', problem_type='multiclass', eval_metric='accuracy')
    return predictor.fit(
        train_data=train_data,
        tuning_data=val_data,
        hyperparameters={
            'NN_TORCH': {
                'batch_size': ag.space.Categorical(2, 4),
                'num_epochs': ag.space.Categorical(1, 3, 5),
                'learning_rate': ag.space.Categorical(1e-3, 1e-5),
                'num_layers': 1,
                'hidden_size': 16,
                'activation': 'relu'
            }
        },
        hyperparameter_tune_kwargs={
            'num_trials': 10,
            'scheduler': 'local',
            'searcher': search_method
        },
        num_bag_folds=5,
        num_stack_levels=0,
        use_bag_holdout=True,
        time_limit=100
    )
```

Evaluates the predictor model on the given dataset and shows the metrics and leaderboard stats

```
def evaluate_model(predictor: TabularPredictor, dataset: TabularDataset, description: str):
    """
    Evaluates the predictor on the provided dataset and prints metrics and leaderboard.

    Parameters:
        predictor (TabularPredictor): The model to evaluate.
        dataset (TabularDataset): The dataset for evaluation.
        description (str): A short description to identify the search strategy.
    """
    predictions = predictor.predict(dataset.drop(columns=['target']))
    acc = accuracy_score(true_labels, predictions)
    f1 = f1_score(true_labels, predictions, average='weighted')

    print(description)
    print(f"Test Accuracy: {acc:.4f}")
    print(f"Test F1: {f1:.4f}")

    lb = predictor.leaderboard(dataset)
    print("Leaderboard:")
    print(lb[['model', 'score_test', 'fit_time']])
```

Using the random search strategy

```
# Create and evaluate a predictor using random search
random_pred = build_predictor('random')
evaluate_model(random_pred, test_data, "Random Search:")
```

Outputs

1. System Info

```
===== System Info =====
AutoGluon Version: 1.2
Python Version: 3.11.11
Operating System: Linux
Platform Machine: x86_64
Platform Version: #1 SMP PREEMPT_DYNAMIC Thu Jun 27 21:05:47 UTC 2024
CPU Count: 2
Memory Avail: 10.49 GB / 12.67 GB (82.7%)
Disk Space Avail: 71.16 GB / 107.72 GB (66.1%)
=====
```

```

Autogluon will save models to /content/AutogluonModels/ag-20250226_109303
Train Data Rows: 104
Train Data Columns: 4
Tuning Data Rows: 15
Tuning Data Columns: 4
Label Column: target
Problem Type: multiclass
Preprocessing data ...
Train Data Class Count: 3
Using Feature Generators to preprocess the data ...
Fitting AutoMLPipelineFeatureGenerator...
Available Memory: 10738.08 MB
Train Data (Original) Memory Usage: 0.00 MB (0.0% of available memory)
Inferring data type of each feature based on column values. Set feature_metadata_in to manually specify special dtypes of the features.
Stage 1 Generators:
  Fitting AsTypeFeatureGenerator...
Stage 2 Generators:
  Fitting FillNaFeatureGenerator...
Stage 3 Generators:
  Fitting IdentityFeatureGenerator...
Stage 4 Generators:
  Fitting DropUniqueFeatureGenerator...
Stage 5 Generators:
  Fitting DropDuplicatesFeatureGenerator...
Types of features in original data (raw dtype, special dtypes):
  ('float', []) : 4 | ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Types of features in processed data (raw dtype, special dtypes):
  ('float', []) : 4 | ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
0.1s = Fit runtime
4 features in original data used to generate 4 features in processed data.
Train Data (Processed) Memory Usage: 0.00 MB (0.0% of available memory)
Data preprocessing and feature engineering runtime = 0.11s ...
AutoGluon will gauge predictive performance using evaluation metric: 'accuracy'
To change this, specify the eval_metric parameter of Predictor()
use_bag_holdout=True, will use tuning_data as holdout (will not be used for early stopping).

```

```

use_bag_holdout=True, will use tuning_data as holdout (will not be used for early stopping).
User-specified model hyperparameters to be fit:
{
  'NN_TORCH': [{'batch_size': Categorical(2, 4)], 'num_epochs': Categorical(1, 3, 5)], 'learning_rate': Categorical(0.001, 1e-05)], 'num_layers': 1, 'hidden_size': 16, 'activation': 'relu'}],
}
Fitting 111 models, fit_strategy='sequential' ...
Hyperparameter tuning model: NeuralNetTorch_BAG_L1 ... Tuning model for up to 89.96s of the 99.95s of remaining time.
-----
| Configuration for experiment | NeuralNetTorch_BAG_L1 |
|-----|-----|
| Search algorithm | BasicVariantGenerator |
| Scheduler | FZOScheduler |
| Number of trials | 10 |
|-----|-----|
View detailed results here: /content/AutogluonModels/ag-20250226_109303/models/NeuralNetTorch_BAG_L1
Fitted model: NeuralNetTorch_BAG_L1/4de56_00000 ...
0.4984 = Validation score (accuracy)
21.96s = Training runtime
0.02s = Validation runtime
Fitted model: NeuralNetTorch_BAG_L1/4de56_00001 ...
0.6721 = Validation score (accuracy)
21.33s = Training runtime
0.03s = Validation runtime
Fitted model: NeuralNetTorch_BAG_L1/4de56_00002 ...
0.6346 = Validation score (accuracy)
23.7s = Training runtime
0.06s = Validation runtime
Fitting model: WeightedEnsemble_L2 ... Training model for up to 99.95s of the 9.61s of remaining time.
Ensemble Weights: {'NeuralNetTorch_BAG_L1/4de56_00001': 1.0}
0.6667 = Validation score (accuracy)
0.8s = Training runtime
0.8s = Validation runtime
AutoGluon training complete, total runtime = 90.42s ... Best model: WeightedEnsemble_L2 | Estimated inference throughput: 740.3 rows/s (15 batch size)
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("/content/AutogluonModels/ag-20250226_109303")
2025-02-26 19:04:33,688 ERROR worker.py:422 -- Unhandled error (suppress with 'RAY_IGNORE_UNHANDLED_ERRORS=1'): The worker died unexpectedly while executing this task. Check python-core-worker-*.log files for more information.

Random Search:
Test Accuracy: 0.6452
Test F1 Score: 0.5451
Leaderboard:

```

	model	score	test	fit_time
0	NeuralNetTorch_BAG_L1/4de56_00001	0.645161	21.337453	
1	WeightedEnsemble_L2	0.645161	21.337453	
2	NeuralNetTorch_BAG_L1/4de56_00002	0.612983	23.781852	
3	NeuralNetTorch_BAG_L1/4de56_00000	0.354839	21.958367	

```

# Create and evaluate a predictor using Hyperband Search with Bayesian optimization
bayes_pred = build_predictor('bayes')
evaluate_model(bayes_pred, test_data, "Hyperband Search + Bayes Optimization:")

```



```
Verbosity: 2 (Standard Logging)
AutoGluon Version: 1.2
Python Version: 3.11.11
Operating System: Linux
Platform Machine: amd64
Platform Version: #1 SMP PREEMPT_DYNAMIC Thu Jun 27 21:05:47 UTC 2024
CPU Count: 2
Memory Avail: 16.01 GB / 12.67 GB (79.0%)
Disk Space Avail: 71.16 GB / 107.72 GB (66.1%)

=====
No presets specified! To achieve strong results with AutoGluon, it is recommended to use the available presets. Defaulting to "medium"....
Recommended Presets (for more details refer to https://autogluon.ai/tutorials/tabular/tabular-essentials.html#presets):
presets="experimental" : Best in v1.2: Pre-trained foundation model + parallel fits. The absolute best accuracy without consideration for inference speed. Does not support GPU.
presets="best" : Maximize accuracy. Recommended for most users. Use in competitions and benchmarks.
presets="high" : Strong accuracy with fast inference speed.
presets="good" : Good accuracy with very fast inference speed.
presets="medium" : Fast training time, ideal for initial prototyping.
Warning: Hyperparameter tuning is currently experimental and may cause the process to hang.
Beginning AutoGluon training ... Time Limit = 180s
AutoGluon will save models to "/content/AutoGluonModels/ag-20250226_183225"
Train Data Rows: 104
Train Data Columns: 4
Tuning Data Rows: 15
Tuning Data Columns: 4
Label Column: target
Problem Type: multiclass
Preprocessing data ...
Train Data Class Count: 3
Using Feature Generators to preprocess the data ...
Fitting AutoPipelineFeatureGenerator...
Available Memory: 10249.98 MB
Train Data (Original) Memory Usage: 0.00 MB (0.0% of available memory)
Inferring data type of each feature based on column values. Set feature_metadata_in to manually specify special dtypes of the features.
Stage 1 Generators:
Fitting AsTypeFeatureGenerator...
Stage 2 Generators:
Fitting FillNAFeatureGenerator...
Stage 3 Generators:
Fitting IdentityFeatureGenerator...
Stage 4 Generators:
Fitting DropnaFeatureGenerator...
Stage 5 Generators:
Fitting DropDuplicatesFeatureGenerator...
Types of features in original data (raw dtype, special dtypes):
('float', []) : 4 | ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Types of features in processed data (raw dtype, special dtypes):
('float', []) : 4 | ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
0.8s = Fit runtime
4 features in original data used to generate 4 features in processed data.
Train Data (Processed) Memory Usage: 0.00 MB (0.0% of available memory)
Data preprocessing and feature engineering runtime = 0.86s ...
AutoGluon will gauge predictive performance using evaluation metric: 'accuracy'
To change this, specify the eval_metric parameter of Predictor().
use_bag_holdout=True, will use tuning_data as holdout (will not be used for early stopping).
User-specified model hyperparameters to be fit:
{
  'NN_TORCH': [{'batch_size': Categorical[2, 4], 'num_epochs': Categorical[1, 3, 5], 'learning_rate': Categorical[0.001, 1e-05], 'num_layers': 1, 'hidden_size': 16, 'activation': 'relu'}],
}
Fitting 1 L1 models, fit_strategy="sequential" ...
Hyperparameter tuning model: NeuralNetTorch_BAG_L1 ... Tuning model for up to 89.94s of the 99.93s of remaining time.
+-----+-----+
| Configuration for experiment | NeuralNetTorch_BAG_L1 |
+-----+-----+
| Search algorithm | SearchGenerator |
| Scheduler | FIFOSScheduler |
| Number of trials | 10 |
+-----+-----+

View detailed results here: /content/AutoGluonModels/ag-20250226_190433/models/NeuralNetTorch_BAG_L1
Fitted model: NeuralNetTorch_BAG_L1/869d9261 ...
0.4904 = Validation score (accuracy)
20.87s = Training runtime
0.83s = Validation runtime
Fitted model: NeuralNetTorch_BAG_L1/70637083 ...
0.1346 = Validation score (accuracy)
21.45s = Training runtime
0.83s = Validation runtime
Fitted model: NeuralNetTorch_BAG_L1/d273a16d ...
0.625 = Validation score (accuracy)
21.0s = Training runtime
0.83s = Validation runtime
Fitting model: WeightedEnsemble_L2 ... Training model for up to 99.93s of the 9.56s of remaining time.
Ensemble Weights: {'NeuralNetTorch_BAG_L1/d273a16d': 1.0}
0.5333 = Validation score (accuracy)
0.8s = Training runtime
0.8s = Validation runtime
AutoGluon training complete, total runtime = 98.47s ... Best model: WeightedEnsemble_L2 | Estimated inference throughput: 703.9 rows/s (15 batch size)
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("/content/AutoGluonModels/ag-20250226_190433")

Hyperband Search + Bayes Optimization:
Test Accuracy: 0.6129
Test F1 Score: 0.5192
Leaderboard:
+-----+-----+-----+-----+
| | model | score_test | fit_time |
+-----+-----+-----+-----+
0 | NeuralNetTorch_BAG_L1/d273a16d | 0.612983 | 20.996867 |
1 | WeightedEnsemble_L2 | 0.612983 | 21.000412 |
2 | NeuralNetTorch_BAG_L1/869d9261 | 0.354839 | 20.872975 |
3 | NeuralNetTorch_BAG_L1/70637083 | 0.032258 | 21.450778 |
```

Some Observations

✓ Impact of Hyperparameters on Model Performance

Epochs: Increasing the number of epochs generally improves model performance because it allows the model more iterations to learn from the data. This trend is visible in the loss-versus-epochs plots, where a higher epoch count corresponds with lower loss values, indicating better learning.

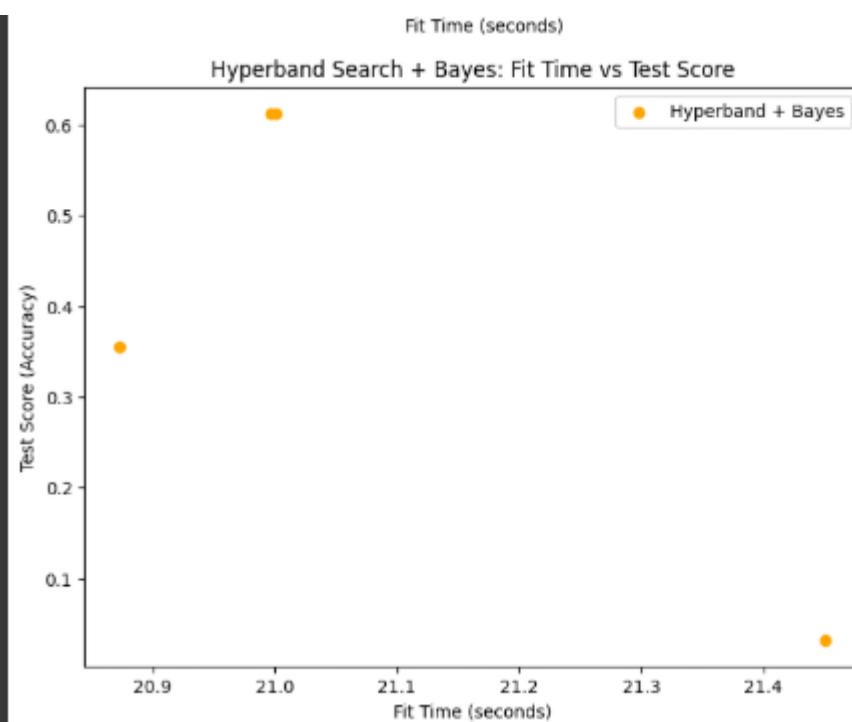
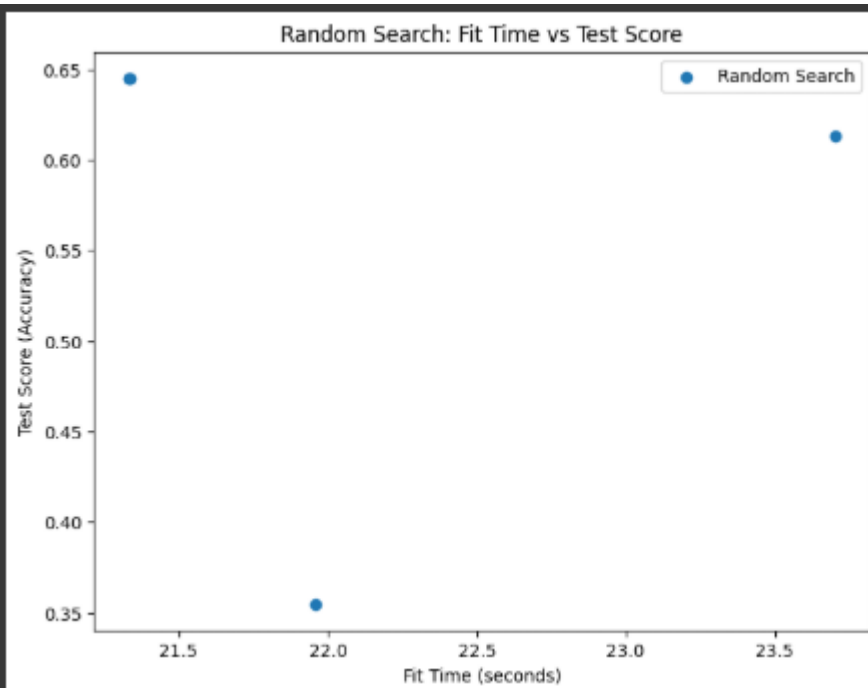
Batch Size: A smaller batch size tends to enhance performance, as it leads to more frequent weight updates during training. In contrast, larger batch sizes may result in higher loss values, suggesting that the model might be less responsive to subtle patterns in the data.

Learning Rate: A higher learning rate can help the model converge faster, often resulting in lower loss values. This relationship is observed across different hyperparameter settings, where increasing the learning rate generally correlates with improved performance.

Manual Tuning vs. Automated Search

Automated Search: Automated hyperparameter tuning is particularly advantageous for large, complex models. It efficiently explores a broad range of parameter combinations, often achieving superior results while reducing the time and effort required compared to manual tuning.

Manual Tuning: Manual tuning can be beneficial when you possess strong domain knowledge and intuition about the problem. It may be a more practical option for smaller models or when computational resources are limited, even though it might not explore the parameter space as thoroughly as automated methods.



```
Summary of AutoGluon Search Results:
Search Method Accuracy F1 Score
0 Random Search 0.645161 0.545050
1 Hyperband + Bayes 0.612903 0.519201
```